

| Aspect          | Virtual Threads (Preview)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version    | Java 19                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Year            | 2022                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Feature         | Virtual threads (Preview)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| UniqueClassName | VirtualThreadsPreview                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| What it is      | Lightweight threads managed by the JVM rather than directly mapped one-to-one to OS threads. They were introduced as a <b>preview feature</b> in Java 19 as part of Project Loom.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Main purpose    | Make highly concurrent, thread-per-request applications easier to build by allowing very large numbers of concurrent tasks without requiring a huge number of OS threads.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Advantages      | <p><b>1. Very lightweight:</b> Thousands or even millions of virtual threads can be created compared with a much smaller number of platform threads.</p> <p><b>2. High concurrency:</b> Excellent for applications handling many simultaneous I/O operations.</p> <p><b>3. Simple programming model:</b> Developers can continue using familiar blocking-style code instead of rewriting everything around callbacks/reactive pipelines.</p> <p><b>4. Efficient I/O waiting:</b> A virtual thread can be suspended while waiting for I/O, allowing its carrier/platform thread to execute other work.</p> <p><b>5. Improved scalability:</b> Particularly useful for server applications with many concurrent requests.</p>                         |
| Disadvantages   | <p><b>1. Preview feature:</b> Java 19 required <code>--enable-preview</code>, and the API/behavior was not yet finalized.</p> <p><b>2. Not a CPU-performance solution:</b> Virtual threads don't make CPU-intensive calculations faster.</p> <p><b>3. Pinning concerns:</b> Certain operations, particularly some synchronized/native-code situations in the early implementation, could prevent efficient unmounting from the carrier thread.</p> <p><b>4. Thread-local overhead:</b> Heavy use of <code>ThreadLocal</code> can reduce some of the benefits when creating enormous numbers of threads.</p> <p><b>5. Library compatibility:</b> Older libraries that assume a small, fixed number of platform threads may not behave optimally.</p> |
| When to use     | Use when you have <b>many concurrent, mostly I/O-bound tasks</b> : HTTP requests, database calls, file/network operations, remote service calls, messaging, etc.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| When not to use | Avoid expecting benefits for <b>CPU-bound workloads</b> such as                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |

|                                             |                                                                                                                                                                                                                                                  |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                             | intensive mathematical calculations, image processing, compression, or large data transformations. Also avoid relying on Java 19 virtual threads in production where preview features are unacceptable.                                          |
| <b>Real-time use case 1</b>                 | <b>Web/API server:</b> Each incoming HTTP request can run in its own virtual thread. If 100,000 requests are waiting on databases or external APIs, virtual threads can provide much greater concurrency than creating 100,000 platform threads. |
| <b>Example 1</b>                            | <code>Thread.startVirtualThread(() -&gt; handleRequest());</code> — starts a task on a virtual thread.                                                                                                                                           |
| <b>Real-time use case 2</b>                 | <b>Microservices:</b> A service making several external REST calls can handle many concurrent requests while virtual threads spend most of their time waiting for network responses.                                                             |
| <b>Example 2</b>                            | <pre>try (var executor =     Executors.newVirtualThreadPerTaskExecutor()) {     executor.submit(() -&gt; callRemoteService()); }</pre>                                                                                                           |
| <b>Real-time use case 3</b>                 | <b>Database-heavy applications:</b> A large number of requests can independently perform blocking JDBC operations without requiring a huge platform-thread pool.                                                                                 |
| <b>Real-time use case 4</b>                 | <b>Concurrent file/network processing:</b> Applications that maintain many simultaneous connections—such as chat servers, gateways, or integration services—can benefit substantially.                                                           |
| <b>When it is especially valuable</b>       | <b>High-concurrency + blocking I/O + thread-per-task programming model.</b>                                                                                                                                                                      |
| <b>Alternate feature</b>                    | <b>Platform threads (Thread) with ExecutorService</b> — the traditional Java approach. For asynchronous I/O, <b>CompletableFuture</b> or reactive frameworks can also be alternatives.                                                           |
| <b>Key difference from platform threads</b> | A platform thread generally corresponds to an operating-system thread, whereas a virtual thread is a lightweight JVM-managed thread that can be mounted onto and unmounted from carrier threads.                                                 |
| <b>Production note</b>                      | Virtual threads became a <b>final feature in Java 21 (2023)</b> . Therefore, for modern production applications, Java 21+ virtual threads are preferable to the Java 19 preview implementation.                                                  |
| <b>Simple recommendation</b>                | <b>Use:</b> High-concurrency I/O-bound applications. <b>Don't use solely for:</b> CPU-bound performance. <b>For modern production:</b> Prefer Java 21+ rather than the Java 19 preview.                                                          |

|  |  |
|--|--|
|  |  |
|--|--|

| Aspect          | Record Patterns (Preview)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version    | Java 19                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Year            | 2022                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Feature         | Record patterns (Preview)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| UniqueClassName | RecordPatternsPreview                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| What it is      | Record patterns allow Java programs to <b>deconstruct record objects</b> and directly extract their components while performing pattern matching. They were introduced as a <b>preview feature in Java 19</b> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Main purpose    | Reduce repetitive accessor calls such as <code>person.name()</code> and <code>person.age()</code> when checking and extracting data from records.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Advantages      | <ol style="list-style-type: none"> <li><b>1. Concise code:</b> Extracts record components directly.</li> <li><b>2. Better readability:</b> The structure of the record is visible in the pattern.</li> <li><b>3. Less boilerplate:</b> Reduces repeated accessor calls.</li> <li><b>4. Works naturally with pattern matching:</b> Can be combined with <code>instanceof</code> and, in later Java versions, <code>switch</code>.</li> <li><b>5. Supports nested deconstruction:</b> Components that are themselves records can be deconstructed further.</li> </ol>                                                                                                              |
| Disadvantages   | <ol style="list-style-type: none"> <li><b>1. Preview feature in Java 19:</b> Required preview enablement and was subject to change.</li> <li><b>2. Record-specific:</b> Primarily useful with records, so it doesn't replace ordinary object-oriented techniques.</li> <li><b>3. Can become complex:</b> Deeply nested record patterns may reduce readability.</li> <li><b>4. Requires familiarity with pattern matching:</b> Developers unfamiliar with modern Java syntax may initially find it harder to understand.</li> <li><b>5. Not necessarily useful for simple records:</b> For straightforward access, ordinary accessor methods can sometimes be clearer.</li> </ol> |
| When to use     | Use when you need to <b>check the type of a record and extract several components at the same time</b> , especially when processing immutable data structures.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| When not to use | Avoid when working mostly with ordinary classes, when only one component is required, or when a complex nested pattern makes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |

|                                           |                                                                                                                                                                                                    |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                           | the code harder to understand. Also avoid the Java 19 preview version for production systems that cannot rely on preview features.                                                                 |
| <b>Real-time use case 1</b>               | <b>Customer data processing:</b> Extract customer name and age while checking that an object is a Customer record.                                                                                 |
| <b>Example 1</b>                          | <pre>if (obj instanceof Customer(String name, int age)) {     System.out.println(name + " - " + age); }</pre>                                                                                      |
| <b>Real-time use case 2</b>               | <b>API response processing:</b> Records can represent immutable API response data, and record patterns can extract fields directly.                                                                |
| <b>Example 2</b>                          | <pre>if (response instanceof ApiResponse(int status, String message)) {     System.out.println(status + ": " + message); }</pre>                                                                   |
| <b>Real-time use case 3</b>               | <b>Nested data processing:</b> A record containing another record can be deconstructed in a single pattern.                                                                                        |
| <b>Example 3</b>                          | <pre>record Address(String city, String country) {} record Customer(String name, Address address) {} if (obj instanceof Customer(String name, Address(String city, String country))) { ... }</pre> |
| <b>Real-time use case 4</b>               | <b>Event/message processing:</b> An application can represent events as records and extract their contents while determining their type.                                                           |
| <b>Example 4</b>                          | <pre>if (event instanceof OrderCreated(long id, String customer)) {     process(id, customer); }</pre>                                                                                             |
| <b>When it is especially valuable</b>     | <b>Immutable data + records + type checking + component extraction</b> , particularly in data-processing and event-driven applications.                                                            |
| <b>Alternate feature</b>                  | <b>Traditional record accessors + instanceof pattern matching.</b><br>For example: <pre>if (obj instanceof Customer c) { String name = c.name(); int age = c.age(); }</pre>                        |
| <b>Alternative before record patterns</b> | Without record patterns, developers had to first match the record and then explicitly call its accessor methods: <pre>if (obj instanceof Customer c) { System.out.println(c.name()); }</pre>       |
| <b>Java 19 status</b>                     | <b>Preview</b> — it was not a permanent Java language feature yet and required preview support.                                                                                                    |
| <b>Later evolution</b>                    | Record patterns were refined in <b>Java 20</b> and became <b>final in Java 21</b> .                                                                                                                |
| <b>Modern recommendation</b>              | For current production code, prefer <b>Java 21+ record patterns</b> , rather than the Java 19 preview implementation.                                                                              |

|                              |                                                                                                                                                                                                      |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                              |                                                                                                                                                                                                      |
| <b>Simple recommendation</b> | <b>Use:</b> When records need convenient deconstruction and pattern matching. <b>Don't use:</b> When it makes simple code more complicated or when you're restricted to finalized language features. |